

A 1.3.1-1.3.5 Operating Systems



A 1.3 Operating Systems and Control Systems

Every program you run depends on a layer of software you almost never see. The operating system sits between your applications and the hardware, deciding which program runs next, where files are kept, and how each device is reached, while hiding the hardware detail that would otherwise make every program far harder to write. These five statements build on each other: they start with what the operating system is for, move through the functions it performs, and then look closely at two decisions it makes constantly, how to schedule work and how to notice when a device needs attention. The higher level statement extends this to how one machine juggles many tasks at once and what goes wrong when those tasks compete for the same resources.

Last Updated: 13 June 2026

Key Questions

- What does it actually mean to say an operating system abstracts the hardware?
- Which functions must an operating system carry out, and what does each one achieve?
- How do the four scheduling approaches differ, and when does each suit a workload?
- When is interrupt handling the better choice than polling, and when can polling be justified?
- (HL) Why does running many tasks on one processor create contention, and how does that lead to deadlock?

IB Command Terms note

This page spans four command terms, and they ask for different things.

- **Describe** (1.3.1, 1.3.2): give a detailed account. For the functions, naming them is not enough; say what each one does.
- **Compare** (1.3.3): give an account of similarities *and* differences, holding the approaches side by side. An answer that describes each algorithm in turn without ever comparing them does not meet the command term.

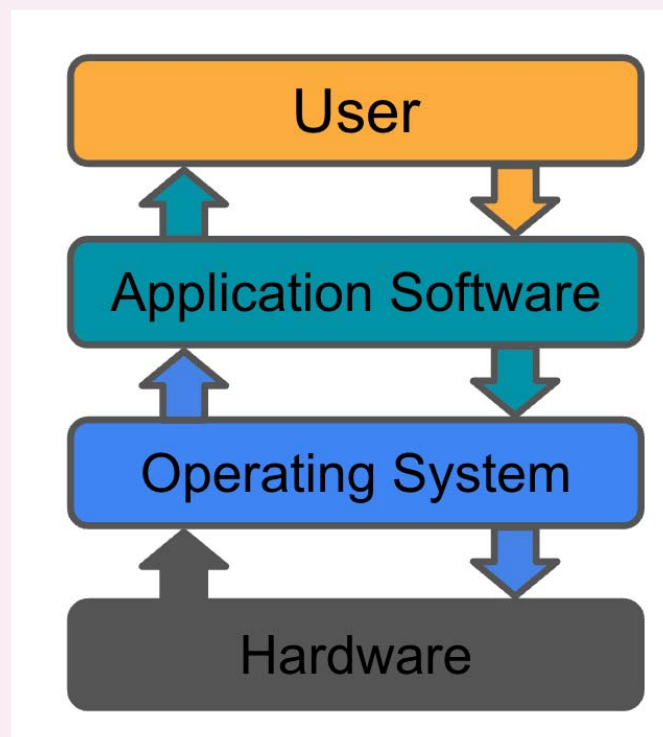
- **Evaluate** (1.3.4): weigh strengths against limitations and reach a judgement. This is the most demanding term here. Listing the pros and cons of polling and interrupts earns some credit; deciding which is better for a stated situation, and saying why, is what "evaluate" requires.
- **Explain** (1.3.5, HL): give a detailed account including reasons and causes. Say *why* multitasking creates the problems it does, not just that it does.

A1.3.1 What is the role of an operating system?

Two ideas sit at the centre of this statement, and they are connected.

The operating system **manages the system's resources**. The processor, main memory and storage are all finite, and several programs usually want them at the same time. The operating system decides which process gets the processor and for how long, hands out memory and reclaims it, and tracks where every file sits on disk. Without these decisions, two programs writing to the same memory or the same file would corrupt each other within seconds.

It does this by **abstracting the hardware**. A program that wants to read a file does not issue the precise electrical commands a particular disk controller expects. It asks the operating system to open a file, and the operating system translates that request into whatever the specific hardware needs. The same program then runs on a machine with a completely different disk, because the messy, device specific part is the operating system's problem, not the program's. Programs are written once against a simple, uniform interface instead of against every possible piece of hardware.

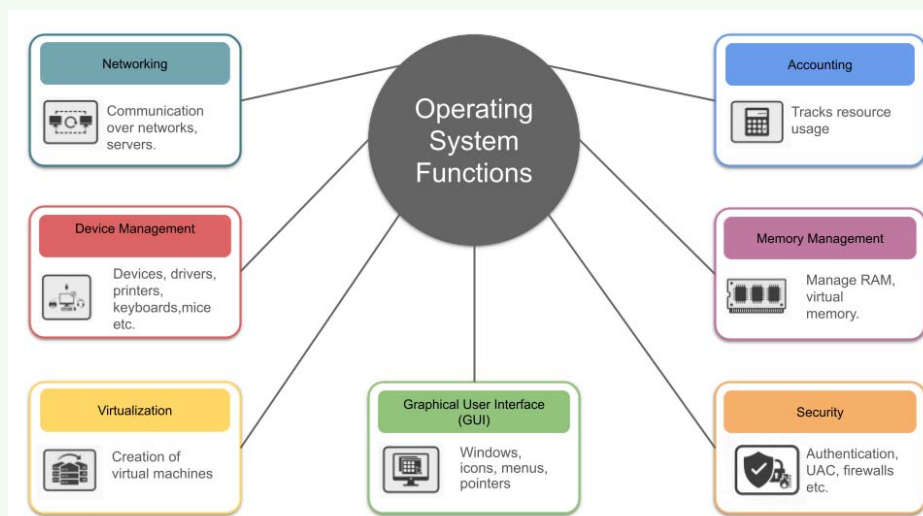


The operating system as an intermediary layer:
requests pass down to the hardware and
results pass back up, with hardware detail
hidden from the software above.

A1.3.2 What functions does an operating system perform?

Much of the operating system's work happens in the background while you do something else. It moves pages of memory between RAM and disk, writes log entries, updates the scheduler's queues and checks permissions, all without any prompt from you.

The point of this constant background activity is to **maintain system integrity**: to keep the system in a consistent, stable state. The syllabus names nine functions through which it does this. Each is summarised below.



The different functions of an operating system.

FUNCTIONS OF AN OPERATING SYSTEM

- **Memory management.** Allocates memory to each process and keeps processes out of each other's space. Virtual memory lets the system behave as though it has more memory than is physically fitted, by moving data that is not currently needed out to disk.
- **File system.** Organises data into files and directories, decides where on the storage device each file physically lives, and controls who may read or change it. When you save a document by name, the file system is what turns that name into blocks on a disk.
- **Device management.** Controls every attached device, from the mouse to the printer, through small pieces of software called drivers. A driver translates a general request, such as "print this page", into the exact instructions one particular printer understands.

- **Scheduling.** Decides which ready process runs on the processor next, and for how long. The approaches to this are the subject of A1.3.3 below.
- **Security.** Protects the system against unauthorised access and malicious software, using authentication (passwords, biometrics), access control over files and resources, and isolation between processes.
- **Accounting.** Records how much of each resource users and processes consume. This supports fair allocation, performance analysis, and billing on shared or cloud systems where usage is charged.
- **Graphical user interface (GUI).** Presents windows, icons, menus and a pointer so that the machine can be used without typing commands. The GUI is itself a program the operating system provides and manages.
- **Virtualisation.** Lets one physical machine host several virtual machines, each behaving like a separate computer with its own operating system.
- **Networking.** Lets the machine communicate with others, handling the packaging, sending, receiving and reassembly of data so that applications can talk across a network without managing the details themselves.

A1.3.3 How do the approaches to scheduling differ?

When several processes are ready at once, only one can use the processor at a time, so the operating system has to choose an order. This changes how long users wait, how evenly work is shared, and how much time is lost switching between tasks. The aim is to allocate processor time so as to get the best performance for the hardware being used.

THINK FIRST

Three programs all want the processor at the same instant: a long video export, a text editor waiting for keystrokes, and a background backup. Before reading on, decide what order you would run them in, and be honest about what "fair" should mean. Most disagreements about scheduling are really disagreements about that word.

The syllabus names four approaches.

First-come first-served (FCFS) runs processes to completion in the order they arrive. It is simple and predictable. Its weakness is the convoy effect: one long job at the front of the queue holds up every short job behind it, pushing up the average waiting time.

Round robin gives each process a fixed time slice, the quantum, then moves it to the back of the queue and starts the next. No process is left waiting indefinitely, which makes the system feel

responsive. The cost is the overhead of switching between processes many times a second, and the quantum has to be chosen with care: too short and switching dominates, too long and it behaves like FCFS.

Multilevel queue scheduling splits processes into separate queues, typically by type or priority, and gives each queue its own rule. Interactive work might sit in a high queue served quickly, while background work waits in a lower queue. This treats different work appropriately, but it is more complex and the risk that a low priority queue is ignored while the higher priority ones stay busy.

Priority scheduling runs the highest priority ready process first. This suits real-time and urgent work, where some tasks genuinely must run before others. Its danger is that a low priority process may never run while higher priority work keeps arriving, unless the system gradually raises the priority of processes that have waited a long time.

Remember for A1.3.3 you must be able to compare. Notice that no approach is best on every way.

Approach	How processor time is allocated	Main strength	Main limitation
First-come first-served	In arrival order, each process runs to completion	Simple and predictable	Convoy effect: a long job delays all jobs behind it
Round robin	Fixed time slice each, then rotate to the back of the queue	Responsive; no process is starved	Frequent context switching adds overhead
Multilevel queue	Processes split into queues by type or priority, each queue with its own rule	Different work gets suitable treatment	Lower queues can starve; more complex to manage
Priority scheduling	Highest priority ready process runs first	Urgent and real-time work served first	Low priority processes can starve without ageing

COMMON EXAM MISTAKE

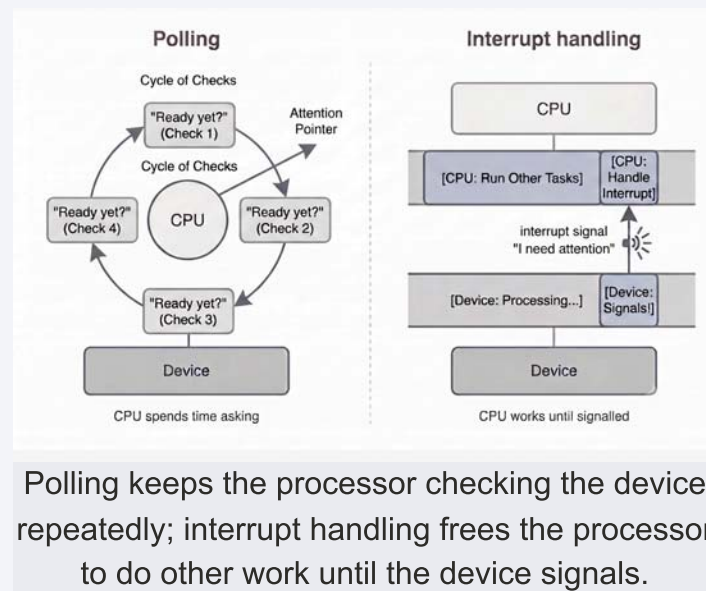
In a "compare" question, candidates often write a paragraph on each algorithm and stop there. That is four descriptions, not a comparison. Pull them together explicitly: say which is more responsive, which carries more overhead, which can cause starvation. The comparison is the bit that scores on this command term.

A1.3.4 Should the system use polling or interrupt handling?

The operating system has to know when a device, such as a keyboard or a network card, has data ready or has finished a job. There are two ways to find out.

With **polling**, the processor repeatedly checks the device and keeps asking if it is ready yet. This is a simple task, but the processor spends time asking even when the answer is almost always no.

With **interrupt handling**, the device sends a signal, an interrupt, when it needs attention. The processor gets on with other work and responds only when the signal arrives. It then saves what it was doing, runs a short interrupt handler, and resumes.



Deciding between them is the "evaluate" part of this statement, and the syllabus lists six considerations that help decide the best approach.

THE SIX CONSIDERATIONS

- **Event frequency.** If a device needs attention almost constantly, polling is reasonable, because the processor would be interrupted continually anyway. If events are rare, interrupts avoid wasting cycles on checks that nearly always find nothing.
- **CPU processing overhead.** Polling consumes processor time even when nothing happens. Interrupts cost almost nothing while the device is idle, but each interrupt carries the overhead of saving and restoring the processor's state, so a flood of interrupts can become expensive.
- **Power source.** On a battery device, a polling loop keeps the processor awake and drains charge. Interrupts let the processor drop into a low power state until something happens, so they are usually preferred on portable hardware. On mains power this matters far less.
- **Event predictability.** If events arrive at known, regular intervals, the system can poll at exactly those moments efficiently. If events are unpredictable, the processor cannot know when to look, which favours interrupts.

- **Controlled latency.** Interrupts usually respond faster to a single event. In a real-time system, though, a steady polling loop can give a more predictable, bounded response time, and predictability is sometimes what matters most.
- **Security concerns.** Interrupt handlers run with high privilege and can be triggered by spurious or malicious signals. In security-sensitive settings, polling gives tighter, more controlled access to a device.

EXAM TIP

"Evaluate" wants a judgement, so apply the considerations to the situation in the question and commit to an answer. Take a keyboard on a battery laptop: keystrokes are infrequent and unpredictable, so polling would burn power checking an idle device, while interrupts let the processor sleep and still respond instantly. The judgement is clear, interrupts win here, and the considerations are your reasons. Contrast that with a busy network card receiving a continuous, predictable stream of packets, where constant interrupts could swamp the processor and periodic polling may actually be more efficient. The same six factors, a different answer, because the situation differs.

A1.3.5 How does the operating system manage multitasking and resource allocation? (HL only)

This statement is **higher level only**.

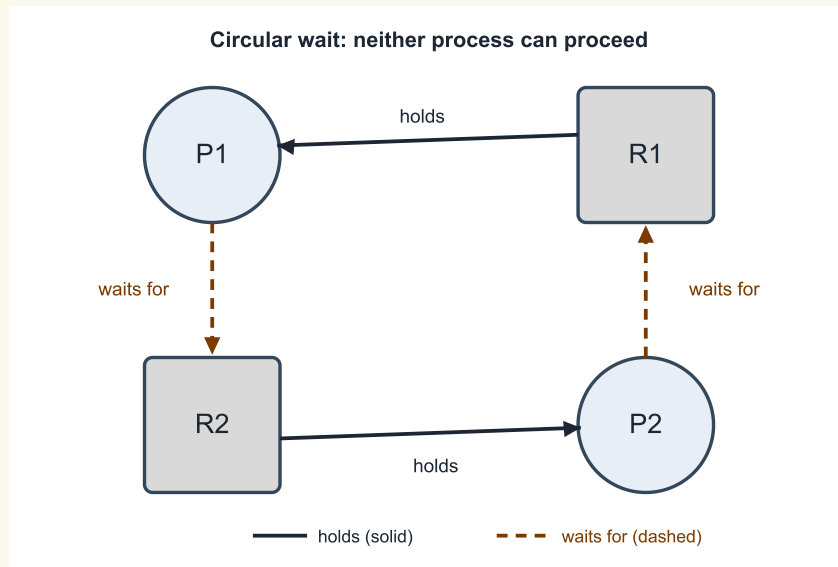
A typical processor core does one thing at a time, yet your machine appears to run a browser, a music player and a download at once. This **illusion of parallelism** comes from the operating system switching between tasks many times a second, giving each a short turn. For each switch the system saves the running task's state (its registers, memory pointers and open files) and restores the next task's. This switching is pure overhead, and doing too much of it wastes the processor it is meant to share.

To keep all this progressing, the operating system acts as the coordinator for every resource: processor time, memory and the input/output devices. Three challenges make that hard.

Task scheduling. With many tasks competing, the system must keep choosing which runs next, using the kinds of approach from A1.3.3. Good scheduling keeps interactive tasks responsive without starving background work.

Resource contention. Two or more tasks often want the same resource, a region of memory, a file, a device, at the same time. The operating system has to arbitrate, which means some tasks wait. Poorly handled, contention causes long delays or lets a high priority task get stuck behind a lower one that holds something it needs.

Deadlock. Contention can reach a state where progress stops entirely. Suppose process P1 holds resource R1 and asks for R2, while P2 holds R2 and asks for R1. Each waits for the other to release what it holds, and neither ever will. This is a circular wait, and it is the classic cause of deadlock.



A circular wait: P1 holds R1 and waits for R2, while P2 holds R2 and waits for R1, so neither process can continue.

Deadlock can occur only when four conditions hold at the same time: **mutual exclusion** (a resource cannot be shared), **hold and wait** (a process holds one resource while waiting for another), **no preemption** (a resource cannot be forcibly taken back) and **circular wait** (the loop above). Because all four are required together, the operating system can prevent deadlock by denying any one of them.

Imposing a fixed order in which resources must be requested, for example, removes the possibility of a circular wait, so the system never gets into the loop in the first place.

COMMON EXAM MISTAKE

Students often say multitasking means the processor runs programs "at the same time". On a single core it does not; it switches between them fast enough to look simultaneous. The distinction matters because the whole reason context switching has a cost, and the reason contention and deadlock exist at all, is that the tasks are really taking turns over shared resources, not running in true parallel.

SUMMARY

Use this summary to help you revise and remember the key points once you have understood all the detail above.

Statement	Key idea	What to be able to do
A1.3.1 Role	Manages resources; abstracts hardware so programs use a simple, uniform interface	Describe both ideas and link them
A1.3.2 Functions	Nine functions: memory management, file system, device management, scheduling, security, accounting, GUI, virtualisation, networking, all keeping the system stable	Name each and say what it does
A1.3.3 Scheduling	FCFS, round robin, multilevel queue, priority; each trades responsiveness, overhead and fairness	Compare them, not just describe
A1.3.4 Polling vs interrupt	Polling checks repeatedly; interrupts signal on demand. Weigh frequency, overhead, power, predictability, latency, security	Reach a judgement for a given case
A1.3.5 Multitasking (HL)	Time-slicing gives the illusion of parallelism; context switching costs overhead; contention can lead to deadlock (four conditions)	Explain the causes and the circular wait

TRY IT ON PAPER

The questions below need written answers. Attempt them on paper (or a laptop if you use one in the exam) before opening the model answers.

1. Compare first-come first-served and round robin scheduling. [4 marks]

- Both are processor scheduling algorithms that decide the order in which ready processes use the processor [1].
- Under FCFS a process runs until it finishes, whereas under round robin a process runs only for a fixed time slice before being moved to the back of the queue [1].
- As a result FCFS can make short processes wait behind a long one (the convoy effect), while round robin shares the processor more evenly so short processes respond sooner [1].
- Round robin pays for this with the overhead of frequent context switching, which FCFS largely avoids [1].

2. Evaluate the use of polling and interrupt handling for taking input from a keyboard on a battery-powered laptop. [6 marks]

- Keystrokes are infrequent and unpredictable: a user may type nothing for minutes, then a burst [1].
- Polling would make the processor check the keyboard repeatedly even when no key is pressed, wasting cycles [1].
- On a battery device that wasted activity drains charge, because the processor cannot drop into a low power state [1].
- Interrupt handling lets the processor sleep or do other work until a key is actually pressed [1].
- The response latency to a single keystroke is low with interrupts, which keeps the keyboard feeling responsive [1].
- Judgement: interrupt handling is the better choice here; its only real cost, the overhead per interrupt, is negligible at typing speeds, so it wins on power use, overhead and responsiveness [1].

3. (HL) Explain how deadlock can arise when two processes share resources. [4 marks]

- Deadlock occurs when two or more processes are each holding a resource and each waiting for a resource the other holds, so none can proceed [1].
- For example, P1 holds R1 and requests R2, while P2 holds R2 and requests R1 [1].
- Neither releases what it holds until it gets what it wants, so both wait indefinitely; this is the circular wait [1].
- It can occur only when mutual exclusion, hold and wait, no preemption and circular wait all hold together, so removing any one prevents it [1].

Operating system Quiz

1

Which of the following is NOT a core function of an operating system?

A. ☐ Running application software

B. ☒ Compiling source code

C. ☐ Providing a user interface

D. ☐ Managing system resources

2

What is the purpose of abstraction in an operating system?

A. ☐ To make the computer run faster

B. ☐ To prevent software crashes

C. ☐ To enable multitasking

D. ☒ To hide complex hardware details from users and applications

3

Which scheduling algorithm gives each process a fixed time slice on the CPU?

A. ☒ Round Robin

B. ☐ First-Come, First-Served

C. ☐ Multilevel Queue Scheduling

D. ☐ Priority Scheduling

4

What is a device driver?

A. ☐ A type of scheduling algorithm

B. ☐ A security feature that prevents unauthorized access

C. ☐ A hardware component that controls a device

D. ☒ A software program that allows the OS to communicate with a device

5

Which of the following is an advantage of virtualization?

A. ☐ Decreased flexibility

B. ☐ Reduced security

C. ☐ Increased hardware costs

D. ☒ Improved resource efficiency

6

What is the main difference between polling and interrupt handling?

A. ☐ Polling is used in real-time systems, while interrupt handling is used in embedded systems

B. ☐ Polling is more efficient than interrupt handling

C. ☐ Interrupt handling is only used for keyboards and mice

D. ☒ Polling involves constantly checking a device, while interrupt handling waits for a signal from the device



7

Which of the following is NOT a factor that influences the choice between polling and interrupts?

A. ☒ The color of the device

- B. ☐ Event frequency
- C. ☐ Power source
- D. ☐ CPU overhead

8

What is deadlock?

- A. ☐ A situation where a program crashes
- B. ☐ A type of scheduling algorithm
- C. ☒ A situation where two or more processes are blocked indefinitely, each waiting for the other
- D. ☐ A security vulnerability

9

Which of the following is a strategy for preventing deadlock?

- A. ☐ Disabling interrupts
- B. ☐ Ignoring resource requests
- C. ☐ Randomly assigning resources
- D. ☒ Resource ordering

10

What is the role of a transducer in a control system?

- A. ☐ To carry out commands from the controller

B. ☐ To gather information about the environment

C. ☐ To make decisions based on sensor input

D. ☐ To convert one form of energy to another

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**